1 Цель работы

Изучить вызов процедур на языке ассемблера из C++ и передачу данных между модулями программы.

2 Постановка задачи

Вариант 7. Написать процедуру на языке ассемблера, которая по исходной строке формирует палиндром с использованием цепочечных команд. Ввод исходной строки и вывод результата выполнить в C++. Данные передавать через аргументы функции по соглашению `cdecl`.

К исходной строке S дописывается её обратная запись S^R. Полученная строка $P = S\,S^R$ читается одинаково в обоих направлениях. Например, `abcd` преобразуется в `abcddcba`.

Программа предназначена для Windows, платформа x86, сборка MSVC и MASM. Допускается ввод до 255 байтов в кодировке CP866. Русская буква в этой кодировке занимает один байт. Пробелы, регистр и знаки препинания сохраняются. Ввод большей длины отклоняется. Для результата выделен буфер из 511 байтов: 510 символов и завершающий нуль.

3 Алгоритм

1. Ввести исходную строку и проверить её длину. Если ввод не выполнен или длина превышает 255 символов, вывести сообщение об ошибке и завершить работу.
2. Передать ассемблерной процедуре адрес исходной строки, адрес буфера результата и длину строки.
3. Скопировать исходную строку в буфер результата.
4. Если строка не пустая, дописать к результату её символы в обратном порядке.
5. Добавить нулевой байт в конец полученной строки, вернуть управление в основную программу и вывести результат.

4 Тестирование

Проверены пустая строка, строки с пробелами, русскими буквами и знаками препинания, а также ввод предельной и превышающей её длины. В таблице приведены исходные данные и полученные результаты. Знак ␣ обозначает пробел, степень — число повторений.

№	Исходная строка	Результат
1	Пустая строка	Пустая строка
2	a	aa
3	ab	abba
4	abc	abccba
5	abcd	abcddcba
6	aba	abaaba
7	aaaa	aaaaaaaa
8	a␣b	a␣bb␣a
9	␣a␣	␣a␣␣a␣
10	Ab1!	Ab1!!1bA
11	тест	тесттсет
12	a␣роза	a␣розаазор␣a
13	$(\mathtt{abc})^{85}$, 255 символов	$(\mathtt{abc})^{85}(\mathtt{cba})^{85}$, 510 символов
14	$\mathtt{a}^{256}$	Ошибка: превышена длина ввода
15	$\mathtt{b}^{300}$	Ошибка: превышена длина ввода
16	1 пробел	2 пробела
17	3 пробела	6 пробелов
18	Нет строки (EOF)	Ошибка ввода

Во всех приведённых примерах результат совпал с ожидаемым.

На рисунке 1 показан пример работы программы с русской строкой.

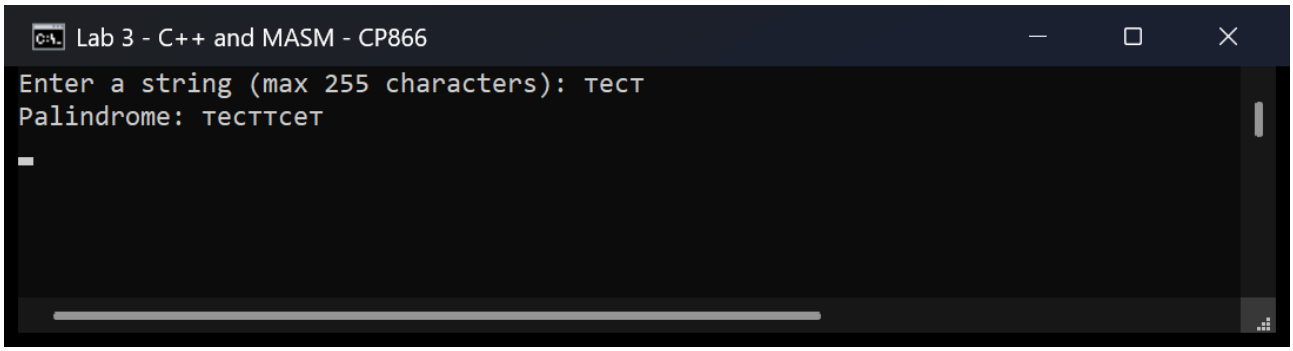


Рисунок 1 — Ввод и вывод русской строки в CP866

5 Код программы

Модуль C++ — main.cpp

```
#include <iostream>
#include <string>
#include <windows.h>

extern "C" void __cdecl MakePalindrome(
    const char* source, char* result, int length);

int main() {
    SetConsoleCP(866);
    SetConsoleOutputCP(866);

    const int MAX_LENGTH = 255;
    std::string source;
    char result[2 * MAX_LENGTH + 1];

    std::cout << "Enter a string (max 255 characters): ";
    if (!std::getline(std::cin, source)) {
        std::cerr << "Error: input failed.\n";
        return 1;
    }
    if (source.size() > MAX_LENGTH) {
        std::cerr << "Error: string is longer than 255 characters.\n";
        return 1;
    }

    MakePalindrome(source.c_str(), result, static_cast<int>(source.size()));
    std::cout << "Palindrome: " << result << '\n';
    return 0;
}
```

Модуль MASM — palindrome.asm

```
.386
.MODEL FLAT
PUBLIC _MakePalindrome

.CODE
; MakePalindrome(const char* source, char* result, int length), cdecl.
_MakePalindrome PROC
    PUSH EBP
    MOV EBP, ESP
    PUSH ESI
    PUSH EDI

    ; Копируем исходную строку в начало результата.
    CLD
    MOV ESI, [EBP+8]
    MOV EDI, [EBP+12]
    MOV ECX, [EBP+16]
    REP MOVSB

    ; Читаем исходную строку с конца и дописываем её копию.
    MOV ECX, [EBP+16]
    JECXZ palindromeReady
    MOV ESI, [EBP+8]
    ADD ESI, ECX
    DEC ESI
copyBackwards:
    STD
    LODSB
    CLD
    STOSB
    LOOP copyBackwards

palindromeReady:
    MOV BYTE PTR [EDI], 0
    POP EDI
    POP ESI
    POP EBP
    RET
_MakePalindrome ENDP
END
```

6 Вывод

Для вызова ассемблерной процедуры из C++ использованы три аргумента: адрес исходной строки, адрес результата и длина. При `cdecl` вызывающий код освобождает 12 байтов аргументов, поэтому в процедуре достаточно `RET` без операнда. Регистры `ESI` и `EDI`, используемые цепочечными командами, восстанавливаются перед возвратом. Сочетание `STD; LODSB; CLD; STOSB` позволяет читать строку с конца и одновременно записывать её обратную копию вперёд.